

Exercise 2 Recurrence

Due date: 2025-11-19

Tasks: 3

This exercise introduces recurrent models with a special focus on modeling dynamical systems. In the first part you will train recurrent neural networks (RNNs) for dynamical systems, and in the second part you will learn about a method called neuralODE.

We will first introduce the two dynamical systems considered in this exercise, and afterwards discuss the programming tasks.

1 Van Der Pol system

Van der Pol is a non-conservative oscillatory system with a nonlinear damping term to describe oscillations in a vacuum tube electrical circuit. The system is described as a two dimensional ODE

$$\begin{cases} \dot{y}_1 &= y_2 \\ \dot{y}_2 &= \mu(1 - y_1^2) - y_1 + y_2, \end{cases}$$

where μ is a scalar parameter indicating the nonlinearity and the strength of the damping. In this exercise we set $\mu = 1$, that is, in the limit cycle regime (periodic behaviors).

2 Lorenz system

The Lorenz equations model atmospheric convection and serves as a typical example of a chaotic system. This system has parameters σ, ρ, β , which control the dynamics of the system given by

$$\begin{cases} \dot{y}_1 &= \sigma(y_2 - y_1) \\ \dot{y}_2 &= y_1(\rho - y_3) - y_2 \\ \dot{y}_3 &= y_1 y_2 - \beta y_3 \end{cases}$$

Here, we set $\sigma = 10$, $\beta = \frac{8}{3}$, and $\rho = 28$, which means we are in the chaotic regime.

Task 1/3: RNN for Van der Pol**Points: 15/100**

RNNs are commonly used for time-series forecasting tasks, because they process data recurrently. A recurrent block transforms a given state to the next state, similar to what is done with numerical solvers which can approximate the next state if given the current state. Thus, we consider the simplest idea for modeling ODEs in this exercise, by only using recurrent blocks in our models.

Your task is first to train an RNN with Van der Pol data. The dataset is given in `y_VanDerPol.py` and is already stored as a PyTorch tensor ¹. For the RNN architecture the class `DynRNN` is provided. This class is initialized with two arguments: the dimension and the system name. The dimension refers to the dimension of the state, which is 2 for Van Der Pol, and the system name should be either `'VanDerPol'` (or for the upcoming task `'Lorenz'` — this will allow you to have a different architecture depending on the system).

We recommend to set up the recurrent block as a simple feedforward neural network with 1-2 hidden layers with a *tanh* activation followed by an output layer with no activation. In order for the model to be able to capture the dynamics of a longer sequence in the training loop you should not only predict one step and directly backpropagate with the loss function. Instead, we recommend to loop over the whole timeseries and make predictions, adding the loss terms for each timestep, and to backpropagate once all loss terms have been summed up. A good way to evaluate your model is to use the initial condition (first value in the dataset) and predict all next values in the dataset, i.e. always use the prediction at $n - 1$ as an input to predict the state at n . Once this is plotted, the predicted trajectory should be similar to the true trajectory see figure 1, please plot these to see how your model performs instead of only calculating the MSE loss. When your model seems to be good enough, save the model parameters as `RNN_VanDerPol.pt` and push your code.

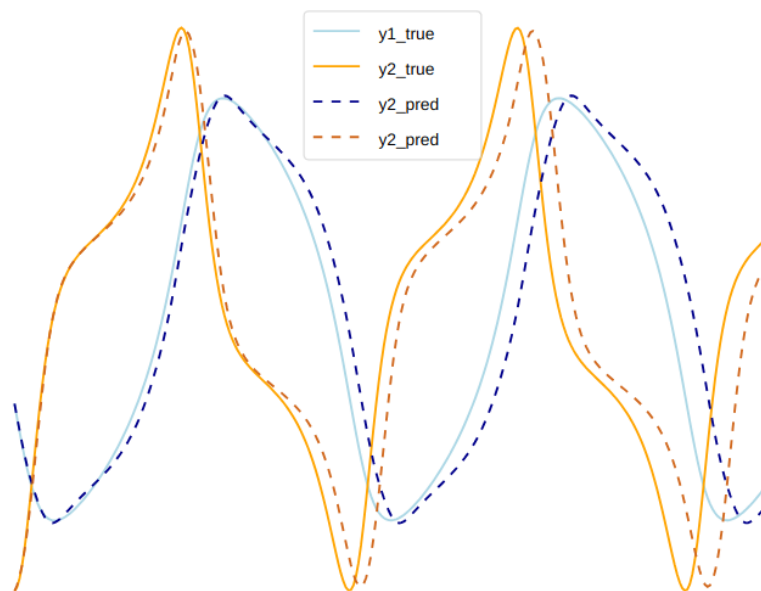


Figure 1: Predicted and true trajectories for Van Der Pol data.

Checklist:

Train a model with Van Der Pol data.

¹Hint: the input data could be `y[:-1]` and the output `y[1:]`

Task 2/3: RNN for Lorenz**Points: 35/100**

Although ODEs can be modelled with simple RNNs, this is not always a good idea. You heard in the lecture that the Jacobian of a system gives insights into the possibility of the gradient vanishing or exploding. In the case of modeling data with an analytically known ODE one can calculate the Jacobian analytically and say that the gradients might explode without doing any training. But also it is possible to calculate the Jacobian using PyTorch's `autograd` module, this would involve calculating the gradients of the (predicted) state n with respect to the state $n - 1$. For the Van der Pol system for example, the Jacobian at timestep n should have a shape 2×2 .

Since this information could be useful when training an RNN, your next task is to implement the calculation of Jacobians in the method `calculate_jacobian` of the `DynRNN` class. The method receives an input tensor, and should return the Jacobians for all timesteps concatenated into a single tensor. Artemis tests will check this implementation, which is expected to work for an arbitrary dimension.

Once you already have this functionality, track the Jacobian for the RNN you trained in the previous task. Do the Jacobians increase with an increasing number of epochs?

Now we proceed to model the chaotic Lorenz system. In the lecture you might have already heard that the inherent properties of this system are such that the Jacobian is large, which is a requirement for the chaotic behavior (due to the relation between the Jacobian and the Lyapunov exponent).

Now you can attempt to train an RNN with Lorenz data, which is available in `y_Lorenz.py`. A similar approach to the one with Van der Pol data can be used here. Again track the Jacobians during training (e.g. track the maximum of the norm over all jacobians). One thing to have in mind is that chaotic systems can be difficult to evaluate with commonly used metrics like the MSE, since a trajectory can behave completely differently due to a very small perturbation (this is also the case for numerical solutions of chaotic systems). Thus, the trajectories typically do not need to match at all timesteps, but rather have a similar behavior / geometry, see for example 2, where both trajectories move along the Lorenz attractor but they do not match completely. We suggest plotting the predicted trajectories or also calculating a power spectrum error, which compares the trajectories in frequency domain. We provide the function `power_spectrum_error` to calculate this in `dynrnn/utils`.

Once you have trained a model, please save your model parameters as `RNN_Lorenz.pt`.

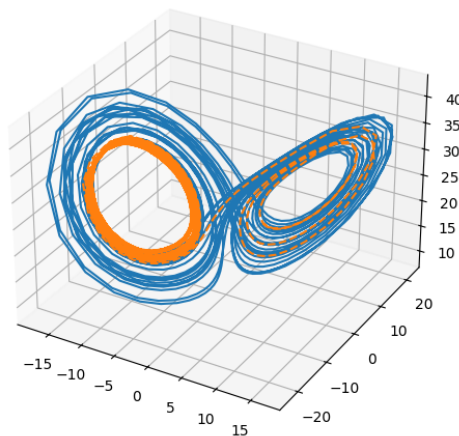


Figure 2: Predicted and true trajectories for Lorenz data.

Checklist:

- Implement `calculate_jacobian`.
- Train a model with Lorenz data.

Task 3/3: NeuralODE**Points: 50/100**

Now we want to train a neural ODE model for a dynamical system. We will consider only Van der Pol system for this part.

A neural ODE learns the right hand side of an ODE. As such, the network can be used as a replacement for the right hand side function of the ODE, and be plugged into an ODE solver. However, the network still needs to be trained. The authors of Neural Ordinary Differential Equations² propose using the so-called adjoint method to solve an additional ODE which provides access to derivatives of a loss w.r.t. the network parameters and allows for updates via backpropagation.

In the folder `neuralode` you will find the necessary functions for training a neural ODE.

A key component of the `backward` method of the class `ODEAdjoint` is missing and thus the provided code cannot properly train a neural ODE. Your first task is to complete the missing part: solve the adjoint ODE for the augmented state. The (initial) augmented state is already available in the variable `aug_state`. Starting from the latest time, use the `odeint` function as an ODE solver, and the function `augmented_dynamics` as a right hand side function. The time values are available in the variable `t`, and since here the ODE is integrated backward in time you should start from the latest time and integrate up until the earliest time. The solver can handle solving from t_n to t_{n-1} such that $t_n > t_{n-1}$, thus you can iterate through time pairs starting from t_N to t_{N-1} , then t_{N-1} to t_{N-2} and so on until the you reach t_0 .

Some technicalities to consider: please pass the `aug_state` as a tuple: `tuple(aug_state)`, and set the parameter `return_last=True` (this ensures that the ODE solver returns the last value only, instead of the result at all provided `t` values). Optionally you can also provide the relative and absolute tolerance values `rtol` and `atol`. The backpropagate function is tested in Artemis.

Once the Adjoint class is complete, you can train a neural ODE. A training setup is given in the `main` file, and you can specify the architecture in `MyNeuralODE` and train a model which gets saved as `'neuralODE_VanDerPol.pt'` and will be evaluated in Artemis.

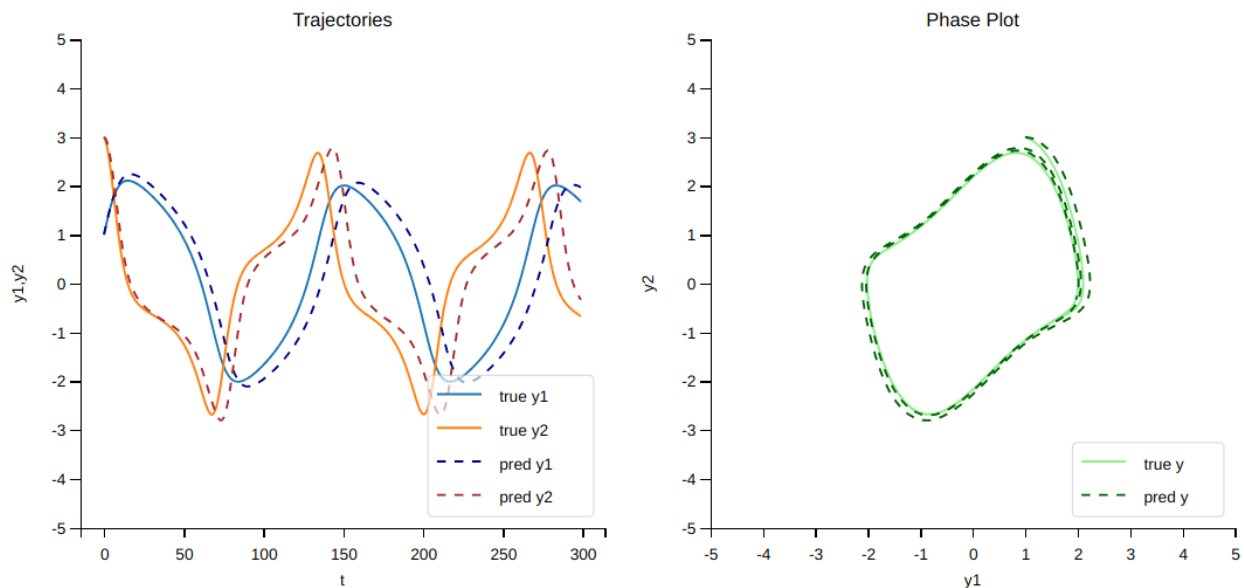


Figure 3: Predicted and true trajectories from a neural ODE for Van der Pol data.

Checklist:

Implement `backward()` method of `ODEAdjoint`.

Train a neural ODE with Van der Pol data.

²<https://arxiv.org/abs/1806.07366>